Centro Paula Souza

Fatec
Jacareí
Prof. Francisco de Moura

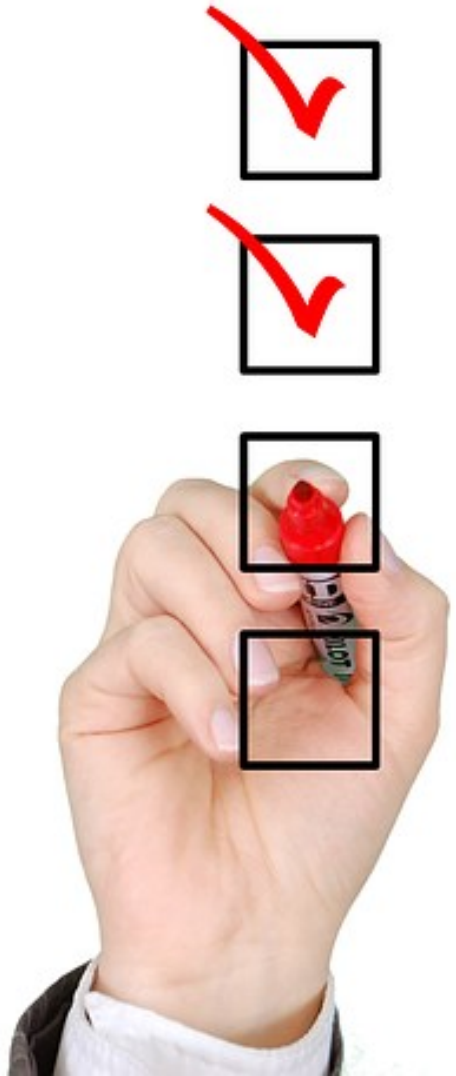
INTEGRAÇÃO E ENTREGA CONTÍNUA

Introdução a Testes Automatizados Avançados

Professora: Lucineide Pimenta

Tópicos da aula

Fatec
Jacareí
Prof. Francisco de Moura



Hoje vamos avançar e ver:

- Revisar o conceito de **testes de integração reais**.
- Aprender a criar **mocks e simulações**.
- Introduzir **testes de segurança automatizados**.
- Integrar relatórios de cobertura e segurança ao pipeline do GitHub Actions.
- Praticar tudo isso em exercícios incrementais.

O que são testes de integração reais?

Fatec
Jacareí
Prof. Francisco de Moura

- Testes de integração garantem que **módulos diferentes** funcionam bem **em conjunto** — não apenas em teoria.
  Enquanto um teste unitário valida uma função isolada, o teste de integração **simula interações reais entre componentes**.
- Exemplo genérico:
  Imagine um sistema de login.
  - O teste unitário garante que verificarSenha() funciona.
  - O teste de integração garante que verificarSenha() + gerarToken() + redirecionarUsuario() funcionam juntos.

Aplicação ao Projeto ABP

Fatec
Jacareí
Prof. Francisco de Moura

- No aplicativo do **INPE – Monitor Climático**, temos funções que se comunicam:

```
function classificarAlerta(valor) {
  if (valor > 80) return "Crítico";
  if (valor > 50) return "Alto";
  return "Moderado";
}

function enviarNotificacao(alerta) {
  return `Notificação enviada: ${alerta}`;
}
```

```
function processarAlerta(valor) {
  const alerta = classificarAlerta(valor);
  return enviarNotificacao(alerta);
}
```

- Nos testes de integração, verificamos **a cadeia completa**:

```
test("processa e envia alerta crítico", () => {
  const resultado = processarAlerta(90);
  expect(resultado).toBe("Notificação enviada: Crítico");
});
```

- Aqui testamos **a jornada do dado**, do cálculo até o envio.

Mocks e simulações

Fatec
Jacareí
Prof. Francisco de Moura

- Nem sempre podemos chamar APIs reais nos testes.
- **Por exemplo:** se o app envia alertas ao servidor do INPE, não queremos disparar alertas reais a cada teste!
- Para isso usamos **mocks**, ou seja, **simulações controladas**.
- **Exemplo:**

```
const api = {
  enviar: jest.fn(() => "Mensagem simulada enviada com sucesso"),
};
```

```
test("envio simulado de alerta", () => {
  const resposta = api.enviar();
  expect(resposta).toBe("Mensagem simulada enviada com sucesso");
});
```

- Assim, testamos o comportamento **sem depender de sistemas externos**.

Testes de Segurança Automatizados

Fatec
Jacareí
Prof. Francisco de Moura

- Agora, vamos além da funcionalidade: precisamos garantir que **o código é seguro**.
- Mesmo um código funcional pode conter falhas como:
- Exposição de tokens e senhas.
- Dependências vulneráveis.
- Falta de validação de dados.
- O GitHub Actions permite usar ferramentas como **npm audit** ou **CodeQL** para detectar vulnerabilidades automaticamente.

- **Exemplo de workflow no .github/workflows/security.yml:**

```
name: Security Check
on: [push, pull_request]
jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Audit dependencies
        run: npm audit --audit-level=moderate
```

- O pipeline falhará se detectar dependências inseguras.

Cobertura de Testes

Fatec
Jacareí
Prof. Francisco de Moura

- Além de escrever testes, precisamos saber **quanto do código foi testado**.
  O Jest tem um comando: *npm test -- --coverage*
- Ele gera um relatório mostrando:
  - Quantas funções foram testadas.
  - Quantas linhas de código foram validadas.
  - O que ainda está sem teste.

- *Isso ajuda a evitar que partes importantes do código fiquem desprotegidas.*

Aplicação ao Projeto ABP

Fatec
Jacareí
Prof. Francisco de Moura

- No app do INPE, esse tipo de verificação é essencial.
  Se um aluno instala um pacote desatualizado, o pipeline avisará:
- "*Vulnerabilidade detectada no pacote axios (versão antiga)!*"
- Isso evita que o projeto final leve **falhas de segurança** para produção.

INTEGRAÇÃO E ENTREGA CONTÍNUA

Atividade Prática (Individual)

Referências para Estudo

Fatec
Jacareí
Prof. Francisco de Moura

- Jest Mocks: https://jestjs.io/docs/mock-functions
- npm audit: https://docs.npmjs.com/cli/v8/commands/npm-audit

INTEGRAÇÃO E ENTREGA CONTÍNUA

Testes de Integração e Segurança Automatizada

Por que precisamos de mais tipos de testes?

Fatec
Jacareí
Prof. Francisco de Moura

- Até agora, usamos apenas **testes unitários** - eles validam **funções isoladas**. Mas, em projetos reais, precisamos ter certeza de que **os módulos funcionam juntos**.
- Pense no app do INPE:
- O **teste unitário** valida que a função *classificarAlerta()* retorna "Crítico" para valores acima de 80.
- O **teste de integração** valida se *classificarAlerta()* + *enviarNotificacao()* funcionam juntos (o alerta é classificado e notificado corretamente).
- O **teste E2E** simula o usuário abrindo o app, preenchendo dados e recebendo a notificação - do começo ao fim.

Estrutura básica de testes no Jest

Fatec
Jacareí
Prof. Francisco de Moura

- Um teste é composto de:
  - **Descrição**: nome do teste (o que será verificado).
  - **Execução**: a ação que queremos testar.
  - **Expectativa**: o resultado esperado.
- Exemplo genérico (unitário):

```
test("soma de dois números", () => {
  expect(2 + 2).toBe(4);
});
```

O que acontece aqui?

1. test("soma de dois números", ... → damos um nome ao teste.
2. expect(2 + 2) → executamos a operação.
3. .toBe(4) → verificamos se o resultado é o esperado.

Exemplo aplicado ao ABP (unitário)

Fatec
Jacareí
Prof. Francisco de Moura

```
function classificarAlerta(valor) {
  if (valor > 80) return "Crítico";
  if (valor > 50) return "Alto";
  return "Moderado";
}

test("classificação de alerta crítico", () => {
  expect(classificarAlerta(90)).toBe("Crítico");
});
```

Explicação:

- *Se o valor for **90**, a função deve retornar "**Crítico**".*
- *O Jest garante que esse comportamento está correto.*

Testes de Integração

Fatec
Jacareí
Prof. Francisco de Moura

- Agora vamos unir funções.
- **Exemplo aplicado ao ABP:**

```
function classificarAlerta(valor) {
  return valor > 80 ? "Crítico" : "Moderado";
}

function enviarNotificacao(alerta) {
  return `Notificação enviada: ${alerta}`;
}

test("classificação + envio de alerta crítico", () => {
  const alerta = classificarAlerta(90);
  const resultado = enviarNotificacao(alerta);
  expect(resultado).toBe("Notificação enviada: Crítico");
});
```

Aqui testamos se a integração entre duas funções funciona corretamente.

Cobertura de Testes

Fatec
Jacareí
Prof. Francisco de Moura

- Além de escrever testes, precisamos saber **quanto do código foi testado**.
  O Jest tem um comando: *npm test -- --coverage*
- Ele gera um relatório mostrando:
  - Quantas funções foram testadas.
  - Quantas linhas de código foram validadas.
  - O que ainda está sem teste.

- *Isso ajuda a evitar que partes importantes do código fiquem desprotegidas.*

INTEGRAÇÃO E ENTREGA CONTÍNUA

Atividade Prática (Individual)

Referências para Estudo

Fatec
Jacareí
Prof. Francisco de Moura

- Documentação Jest: https://jestjs.io/docs/getting-started
- Vídeo – Introdução a Testes Automatizados: https://www.youtube.com/watch?v=Eu35xM76kKY
- Vídeo – Testes em JavaScript com Jest: https://www.youtube.com/watch?v=FgnxcUQ5vho

O que veremos na próxima aula

Na próxima aula, vamos **explorar:**

- Como interpretar e exibir relatórios de **cobertura e vulnerabilidade** dentro do GitHub Actions.
- Integração dos relatórios com **badges (selos)** no README.

Bibliografia Básica

Fatec
Jacareí
Prof. Francisco de Moura

- HUMBLE J; PRIKLANDNICKI R. **Entrega Contínua:** Como Entregar Software de Forma Rápida e Confiável. São Paulo: Bookman, 2013.
- MUNIZ, A.; et al. **Jornada DevOps:** Unindo Cultura Ágil, Lean e Tecnologia Para Entrega de Software Com Qualidade. São Paulo: Brasport, 2019.
- SATO D. **DevOps na prática:** entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.
- SILVA, R. **Entrega contínua em Android:** Como automatizar a distribuição de apps. São Paulo: Casa do Código, 2016.

Bibliografia Complementar

Fatec
Jacareí
Prof. Francisco de Moura

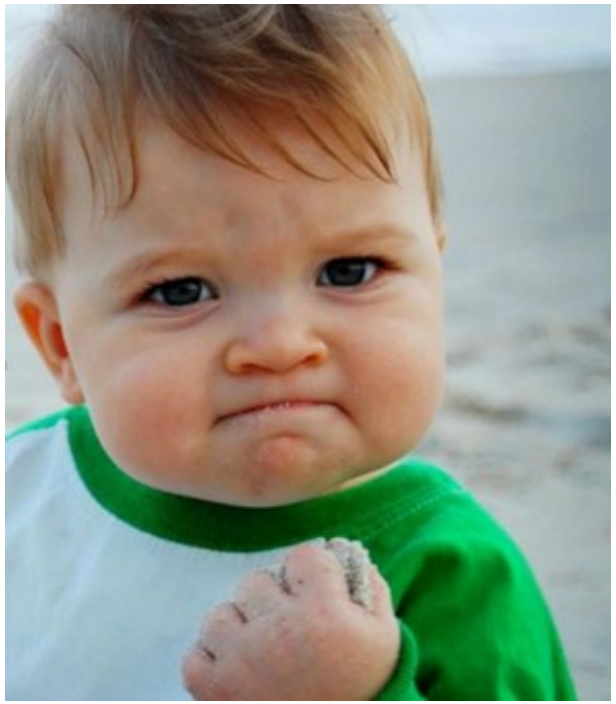
- ARUNDEL, J. DOMINGUS, J. **DevOps nativo de nuvem com Kubernetes**. São Paulo: Novatec, 2019.
- MORAES, G. **Caixa de Ferramentas DevOps:** Um guia para construção, administração e arquitetura de sistemas modernos. São Paulo: Casa do Código, 2015. PIRES, A.; MILITÃO, J. **Integração Contínua com Jenkins**. São Paulo: Casa do Código, 2019.
- VITALINO, J. F. N.; CASTRO, M. A. N. **Descomplicando o Docker.** 2 ed. São Paulo: Brasport, 2018.
- SILVERMAN, R. E. **Git:** guia prático**.** São Paulo: Novatec, 2019.

Dúvidas?

Fatec
Jacareí
Prof. Francisco de Moura

Considerações Finais

Fatec
Jacareí
Prof. Francisco de Moura



Professora:

Lucineide Pimenta

Bom descanso à todos!